BITS Pilani
K K Birla Goa Campus

INSTRUCTION SET

MICROPROCESSORS & INTERFACING (CS F241)

Dr. Gargi Alavani
Department of CS & IS

innovate achieve lead

Arithmetic Instructions Learnt So far…

Number Systems

- Negative numbers are stored as 2's complement format
- Subtraction in 8086 is done using 2's complement

E.g.
MOV CH,22H
SUB CH,44H

```
22H        00100010
44H      + 10111100
           11011110
```

$$Z = 0 \text{ (result not zero)}$$
$$C = 1 \text{ (borrow)}$$
$$A = 1 \text{ (half-borrow)}$$
$$S = 1 \text{ (result negative)}$$
$$P = 1 \text{ (even parity)}$$
$$O = 0 \text{ (no overflow)}$$

Number Systems

- Negative numbers are stored as 2's complement format
- Subtraction in 8086 is done using 2's complement

E.g.
MOV CH,22H
SUB CH,44H

22H 00100010
44H + 10111100
11011110

- AF is set if there is a borrow from bit 3 to bit 4 during the subtraction, the AF flag is set.

BCD & ASCII Arithmetic

ASCII

- American Standard Code for Information Interchange
- Uses a 7-bit binary code to represent characters, including letters (both uppercase and lowercase), numbers, punctuation marks, and control characters.

ASCII

Hex	Value	Hex	Value	Hex	Value	Hex	Value	Hex	Value	Hex	Value	Hex	Value	Hex	Value
00	NUL	10	DLE	20	SP	30	0	40	@	50	P	60	`	70	p
01	SOH	11	DC1	21	!	31	1	41	A	51	Q	61	a	71	q
02	STX	12	DC2	22	"	32	2	42	B	52	R	62	b	72	r
03	ETX	13	DC3	23	#	33	3	43	C	53	S	63	c	73	s
04	EOT	14	DC4	24	$	34	4	44	D	54	T	64	d	74	t
05	ENQ	15	NAK	25	%	35	5	45	E	55	U	65	e	75	u
06	ACK	16	SYN	26	&	36	6	46	F	56	V	66	f	76	v
07	BEL	17	ETB	27	'	37	7	47	G	57	W	67	g	77	w
08	BS	18	CAN	28	(	38	8	48	H	58	X	68	h	78	x
09	HT	19	EM	29	)	39	9	49	I	59	Y	69	i	79	y
0A	LF	1A	SUB	2A	*	3A	:	4A	J	5A	Z	6A	j	7A	z
0B	VT	1B	ESC	2B	+	3B	;	4B	K	5B	[	6B	k	7B	{
0C	FF	1C	FS	2C	,	3C	<	4C	L	5C	\	6C	l	7C	|
0D	CR	1D	GS	2D	-	3D	=	4D	M	5D	]	6D	m	7D	}
0E	SO	1E	RS	2E	.	3E	>	4E	N	5E	^	6E	n	7E	~
0F	SI	1F	US	2F	/	3F	?	4F	O	5F	_	6F	o	7F	DEL

ASCII Arithmetic

- ASCII Arithmetic instructions function with ASCII-coded numbers.
- 0-9 -> 30H to 39H

Four Instructions
- AAA (ASCII adjust after Addition)
- AAD (ASCII adjust before Division)
- AAM (ASCII adjust after Multiplication)
- AAS (ASCII adjust after Subtraction)

E.g.
MOV AX,31H
ADD AL, 39H
AAA
ADD AX,3030H

ASCII

```
0000    42 61 72 72 79    NAMES   DB   'Barry B. Brey'
        20 42 2E 20 42
        72 65 79
OOOD    57 68 65 20 63    MESS    DB   'Where can it be?'
        20 63 61 6E 20
        69 74 20 62 65
        3F
001D    57 69 20 74 20    WHAT    DB   'What is on first.'
        69 73 20 6F 6E
        20 66 69 72 73
        74 2E
```

AAA Instruction

- The AAA instruction adjusts the contents of the AL (accumulator low) register based on the result of a previous addition. Specifically, it performs the following steps:

1. Checks if the low nibble (bits 0-3) of AL contains a decimal value between 0 and 9 or if the Auxiliary Carry Flag (AF) is set.
2. If the low nibble is within the range of 0 to 9 or the AF is set, no adjustment is needed, and the instruction proceeds to step 5.
3. If the low nibble is in the range 10 to 15, or the AF is not set, the following adjustments are made:
    - AL is incremented by 6.
    - AH (accumulator high) is incremented by 1.
    - The Carry Flag (CF) and AF are set.
4. OR The AF and CF are cleared.
5. The contents of the high nibble (bits 4-7) of AL are cleared

Example

E.g.
MOV AX,31H
ADD AL, 39H
AAA
ADD AX,3030H

The AAA instruction clears AH if the result is less than 10, and adds 1 to AH if the result is greater than 10.

```
AX=0031 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0859 ES=0859 SS=0859 CS=0859 IP=0103 NV UP EI NG NZ NA PE NC
0859:0103 0439              ADD     AL,39
-
AX=006A BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0859 ES=0859 SS=0859 CS=0859 IP=0105 NV UP EI PL NZ NA PE NC
0859:0105 37                AAA
-
AX=0100 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0859 ES=0859 SS=0859 CS=0859 IP=0106 NV UP EI PL NZ AC PO CY
0859:0106 053030            ADD     AX,3030
-
AX=3130 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0859 ES=0859 SS=0859 CS=0859 IP=0109 NV UP EI PL NZ NA PE NC
0859:0109 0000              ADD     [BX+SI],AL
-
```

Example

```
if low nibble of AL > 9 or AF = 1 then:
    AL = AL + 6
    AH = AH + 1
    AF = 1
    CF = 1
else
    AF = 0
    CF = 0
in both cases:
    clear the high nibble of AL.
```

```
MOV AX,31H
ADD AL, 39H
AAA
ADD AX,3030H
```

```
  0011 0001
+ 0011 1001
  0110 1010
-------------
     6    A
```

AL=6A+6
AH=0+1
AF=1 CF=1

What about AAA after 39H + 39H ?

BCD

- Binary Coded Decimal. It is a way of representing decimal numbers in a binary form, where each decimal digit is represented by a fixed number of binary bits.
- In BCD, each decimal digit is encoded using a 4-bit binary representation.

  Decimal – 12

  Binary – 1100

  BCD – 0001 0010
- BCD is often used in digital systems where decimal arithmetic is required, such as in calculators and some embedded systems.
- It allows for easy conversion between binary and decimal representations.

DAA Instruction

```
old_AL := AL;
old_CF := CF;
CF := 0;
IF (((AL AND 0FH) > 9) or AF = 1)
        THEN
            AL := AL + 6;
            CF := old_CF or (Carry from AL := AL + 6);
            AF := 1;
        ELSE
            AF := 0;
FI;
IF ((old_AL > 99H) or (old_CF = 1))
    THEN
            AL := AL + 60H;
            CF := 1;
    ELSE
            CF := 0;
FI;
```

BCD Arithmetic

- DAA (Decimal Adjust after addition)
- DAS (Decimal adjust after subtraction)
- Both the instruction correct the result so that result is also BCD

E.g.

```
MOV DX,1234H
MOV BX, 3099H
MOV AL,BL
ADD AL,DL
DAA
```

Example

```
0859:0100 BA3412          MOV     DX,1234
0859:0103 BB9930          MOV     BX,3099
0859:0106 88D8            MOV     AL,BL
0859:0108 00D0            ADD     AL,DL
0859:010A 27              DAA
```

E.g.

MOV DX,1234H

MOV BX, 3099H

MOV AL,BL

ADD AL,DL

DAA

```
AX=0000 BX=0000 CX=0000 DX=1234 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0859 ES=0859 SS=0859 CS=0859 IP=0103 NV UP EI NG NZ NA PE NC
0859:0103 BB9930          MOV     BX,3099
-
AX=0000 BX=3099 CX=0000 DX=1234 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0859 ES=0859 SS=0859 CS=0859 IP=0106 NV UP EI NG NZ NA PE NC
0859:0106 88D8            MOV     AL,BL
-
AX=0099 BX=3099 CX=0000 DX=1234 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0859 ES=0859 SS=0859 CS=0859 IP=0108 NV UP EI NG NZ NA PE NC
0859:0108 00D0            ADD     AL,DL
-
AX=00CD BX=3099 CX=0000 DX=1234 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0859 ES=0859 SS=0859 CS=0859 IP=010A NV UP EI NG NZ NA PO NC
0859:010A 27              DAA
-
AX=0033 BX=3099 CX=0000 DX=1234 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0859 ES=0859 SS=0859 CS=0859 IP=010B NV UP EI PL NZ AC PE CY
0859:010B 0000            ADD     [BX+SI],AL
```

Example

```
0000 BA 1234        MOV  DX,1234H       ;load 1234 BCD
0003 BB 3099        MOV  BX,3099H       ;load 3099 BCD
0006 8A C3          MOV  AL,BL          ;sum BL and DL
0008 02 C2          ADD  AL,DL
000A 27             DAA
000B 8A C8          MOV  CL,AL          ;answer to CL
000D 9A C7          MOV  AL,BH          ;sum BH, DH and carry
000F 12 C6          ADC  AL,DH
0011 27             DAA
0012 8A E8          MOV  CH,AL          ;answer to CH
```

Example

```
AX=0033 BX=3099 CX=0000 DX=1234 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0859 ES=0859 SS=0859 CS=0859 IP=010B NV UP EI PL NZ AC PE CY
0859:010B 88C1                MOV     CL,AL
-
AX=0033 BX=3099 CX=0033 DX=1234 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0859 ES=0859 SS=0859 CS=0859 IP=010D NV UP EI PL NZ AC PE CY
0859:010D 88F8                MOV     AL,BH
-
AX=0030 BX=3099 CX=0033 DX=1234 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0859 ES=0859 SS=0859 CS=0859 IP=010F NV UP EI PL NZ AC PE CY
0859:010F 10F0                ADC     AL,DH
-
AX=0043 BX=3099 CX=0033 DX=1234 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0859 ES=0859 SS=0859 CS=0859 IP=0111 NV UP EI PL NZ NA PO NC
0859:0111 27                  DAA
-
AX=0043 BX=3099 CX=0033 DX=1234 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0859 ES=0859 SS=0859 CS=0859 IP=0112 NV UP EI PL NZ NA PO NC
0859:0112 88C5                MOV     CH,AL
-
AX=0043 BX=3099 CX=4333 DX=1234 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0859 ES=0859 SS=0859 CS=0859 IP=0114 NV UP EI PL NZ NA PO NC
0859:0114 0000                ADD     [BX+SI],AL
-
```

DAS

```
old_AL := AL;
old_CF := CF;
CF := 0;
IF (((AL AND 0FH) > 9) or AF = 1)
    THEN
            AL:=AL -6;
            CF := old_CF or (Borrow from AL := AL – 6);
            AF := 1;
    ELSE
            AF := 0;
FI;
IF ((old_AL > 99H) or (old_CF = 1))
        THEN
            AL := AL – 60H;
            CF := 1;
FI;
```

Basic Logic Instructions

- AND, OR, Exclusive-OR, and NOT
- Logic operations provide binary bit control in low-level software. The logic instructions allow bits to be set, cleared, or complemented.
- When binary data are manipulated in a register or a memory location, the rightmost bit position is always numbered bit 0.
- Bit position numbers increase from bit 0 toward the left, to bit 7 for a byte, and to bit 15 for a word.
- A doubleword (32 bits) uses bit position 31 as its leftmost bit and a quadword (64-bits) uses bit position 63 as it leftmost bit.

AND

- The AND operation performs logical multiplication
- 0 AND anything is always 0, and 1 AND 1 is always 1.

A	B	T
0	0	0
0	1	0
1	0	0
1	1	1

AND

Assembly Language	*Operation*
AND AL,BL	AL = AL and BL
AND CX,DX	CX = CX and DX
AND ECX,EDI	ECX = ECX and EDI
AND RDX,RBP	RDX = RDX and RBP (64-bit mode)
AND CL,33H	CL = CL and 33H
AND DI,4FFFH	DI = DI and 4FFFH
AND ESI,34H	ESI = ESI and 34H
AND RAX,1	RAX = RAX and 1 (64-bit mode)
AND AX,[DI]	The word contents of the data segment memory location addressed by DI are ANDed with AX
AND ARRAY[SI],AL	The byte contents of the data segment memory location addressed by ARRAY plus SI are ANDed with AL
AND [EAX],CL	CL is ANDed with the byte contents of the data segment memory location addressed by ECX

Masking

- AND operation clears bits of a binary number , this task is called masking.

```
  x x x x  x x x x    Unknown number
• 0 0 0 0  1 1 1 1    Mask
-------------------
  0 0 0 0  x x x x    Result
```

- The OR instruction uses any of the addressing modes allowed to any other instruction except segment register addressing.

Examples

```
AX=0022 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0859 ES=0859 SS=0859 CS=0859 IP=0103 NV UP EI PL NZ NA PE NC
0859:0103 BB1100              MOV     BX,0011
-
AX=0022 BX=0011 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0859 ES=0859 SS=0859 CS=0859 IP=0106 NV UP EI PL NZ NA PE NC
0859:0106 21D8                AND     AX,BX
-
AX=0000 BX=0011 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0859 ES=0859 SS=0859 CS=0859 IP=0108 NV UP EI PL ZR NA PE NC
0859:0108 3000                XOR     [BX+SI],AL
-
```

Masking Example

- ASCII-coded number can be converted to BCD using masking
- ASCII 30H to 39H -> 0 to 9

```
MOV BX,3135H          ;load ASCII
AND BX,0F0FH          ;mask BX
```

Masking Example

```
MOV BX,3135H          ;load ASCII
AND BX,0F0FH          ;mask BX
```

```
AX=3135 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0859 ES=0859 SS=0859 CS=0859 IP=0103 NV UP EI NG NZ NA PE NC
0859:0103 BB0F0F              MOV     BX,0F0F
-
AX=3135 BX=0F0F CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0859 ES=0859 SS=0859 CS=0859 IP=0106 NV UP EI NG NZ NA PE NC
0859:0106 21D8                AND     AX,BX
-
AX=0105 BX=0F0F CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0859 ES=0859 SS=0859 CS=0859 IP=0108 NV UP EI PL NZ NA PE NC
0859:0108 0000                ADD     [BX+SI],AL
```

OR Gate

- Performs logical addition and is often called the Inclusive-OR function.

A	B	T
0	0	0
0	1	1
1	0	1
1	1	1

Masking with OR

```
    x x x x  x x x x   Unknown number
+   0 0 0 0  1 1 1 1   Mask
-------------------
    x x x x  1 1 1 1   Result
```

OR Examples

Assembly Language	*Operation*
OR AH,BL	AL = AL or BL
OR SI,DX	SI = SI or DX
OR EAX,EBX	EAX = EAX or EBX
OR R9,R10	R9 = R9 or R10 (64-bit mode)
OR DH,0A3H	DH = DH or 0A3H
OR SP,990DH	SP = SP or 990DH
OR EBP,10	EBP = EBP or 10
OR RBP,1000H	RBP = RBP or 1000H (64-bit mode)
OR DX,[BX]	DX is ORed with the word contents of data segment memory location addressed by BX
OR DATES[DI + 2],AL	The byte contents of the data segment memory location addressed by DI plus 2 are ORed with AL

Example

If we multiply two BCD numbers (5,7) , can you convert its result to ASCII?

Example

If we multiply two BCD numbers (5,7) , can you convert its result to ASCII?

AAM

```
tempAL := AL;
AH := tempAL / imm8; (* imm8 is set to 0AH for the AAM mnemonic *)
AL := tempAL MOD imm8;
```

```
MOV   AL,5              ;load data
MOV   BL,7
MUL   BL
AAM                     ;adjust
OR    AX,3030H          ;convert to ASCII
```

BITS Pilani
K K Birla Goa Campus

THANK YOU